

Coin Change - Count Ways to Make Sum

Last Updated : 23 Jul, 2025



Given an integer array of **coins[]** of **size n** representing different types of **denominations** and an integer **sum**, the task is to count all combinations of coins to make a given value **sum**.

Note: Assume that you have an **infinite** supply of each type of coin.

Examples:

Input: *sum = 4, coins[] = [1, 2, 3]*

Output: 4

Explanation: *There are four solutions: [1, 1, 1, 1], [1, 1, 2], [2, 2] and [1, 3]*

Input: *sum = 10, coins[] = [2, 5, 3, 6]*

Output: 5

Explanation: *There are five solutions:*

[2, 2, 2, 2, 2], [2, 2, 3, 3], [2, 2, 6], [2, 3, 5] and [5, 5]

Input: *sum = 10, coins[] = [10]*

Output: 1

Explanation: *The only is to pick 1 coin of value 10.*

Input: *sum = 5, coins[] = [4]*

Output: 0

Explanation: *We cannot make sum 5 with the given coins*

Table of Content

- [\[Naive Approach\] Using Recursion - \$O\(2^{\text{sum}}\)\$ time and \$O\(\text{sum}\)\$ space](#)
- [\[Better Approach 1\] Using Top-Down DP \(Memoization\) - \$O\(\text{sum} * n\)\$ time and \$O\(\text{sum} * n\)\$ space](#)
- [\[Better Approach 2\] Using Bottom-Up DP \(Tabulation\) – \$O\(\text{sum} * n\)\$ time and \$O\(\text{sum} * n\)\$ space](#)
- [\[Expected Approach\] Using Space Optimized DP – \$O\(\text{sum} * n\)\$ time and \$O\(\text{sum}\)\$ space](#)

[Naive Approach] Using Recursion - $O(2^{\text{sum}})$ time and $O(\text{sum})$ space

For each coin, there are 2 options:

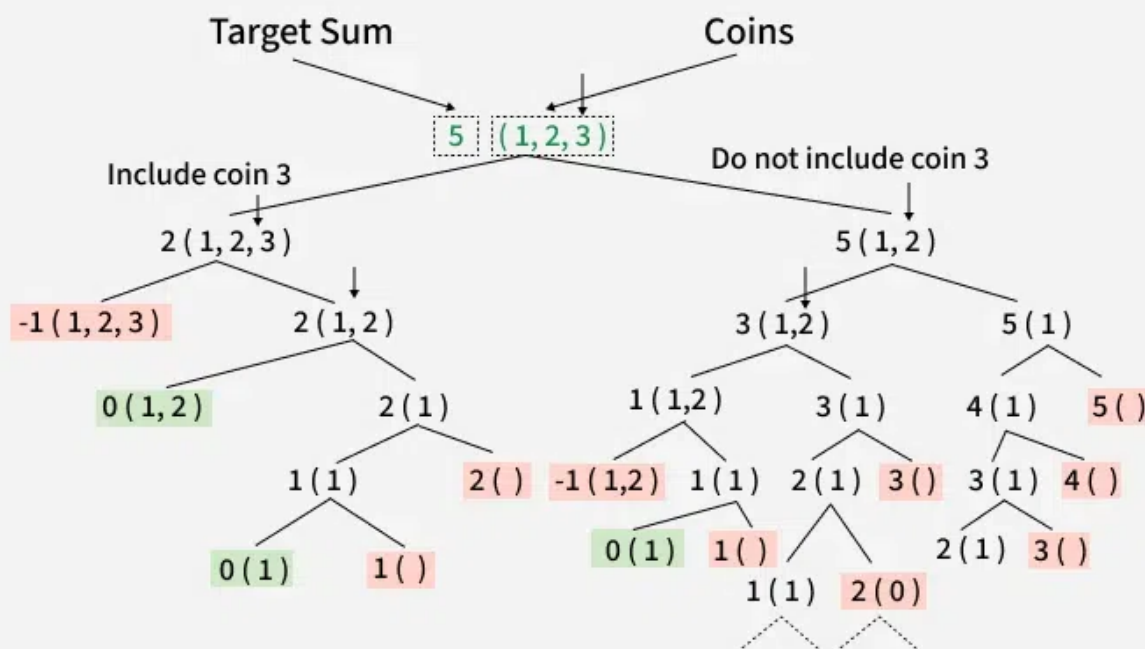
- **Include the current coin:** Subtract the current coin's **denomination** from the **target sum** and call the count function **recursively** with the updated sum and the same set of coins i.e., **count(coins, n, sum - coins[n-1])**
- **Exclude the current coin:** Call the count function **recursively** with the same sum and the remaining coins. i.e., **count(coins, n-1, sum)**.

Recurrence Relation: The final result will be the sum of both cases.

- **$\text{count}(\text{coins}, n, \text{sum}) = \text{count}(\text{coins}, n, \text{sum} - \text{coins}[n-1]) + \text{count}(\text{coins}, n-1, \text{sum})$**

Base cases:

- If the **target sum** (sum) is 0, there is only **one way** to make the sum, which is by not selecting any coin. So, **count(0, coins, n) = 1**.
- If the target sum (sum) is **negative** or no coins are left to consider (**n == coins.size**), then there are **no ways** to make the sum, so **count(sum, coins, 0) = 0**.



C++

Java

Python

C#

JavaScript

```
// C++ program for coin change problem
```

```
// using recursion
#include <bits/stdc++.h>
using namespace std;

// Returns the count of ways we can
// sum coins[0...n-1] coins to get sum "sum"
int countRecur(vector<int>& coins, int n, int sum) {

    // If sum is 0 then there is 1 solution
    // (do not include any coin)
    if (sum == 0) return 1;

    // 0 ways in the following two cases
    if (sum < 0 || n == 0) return 0;

    // count is sum of solutions (i)
    // including coins[n-1] (ii) excluding coins[n-1]
    return countRecur(coins, n, sum - coins[n - 1]) +
           countRecur(coins, n - 1, sum);
}

int count(vector<int> &coins, int sum) {
    return countRecur(coins, coins.size(), sum);
}

int main() {
    vector<int> coins = {1, 2, 3};
    int sum = 5;
    cout << count(coins, sum);
    return 0;
}
```

Output

5

[Better Approach 1] Using Top-Down DP (Memoization) - $O(\text{sum} \times n)$ time and $O(\text{sum} \times n)$ space

1. Optimal Substructure: Number of ways to make **sum** at index **i**, i.e., **count(i, sum, coins)**, depends on the optimal solutions of the subproblems **count(i, sum-coins[i-1], coins)** , and **count(i-1, sum, coins)**. By adding these optimal substructures, we can efficiently calculate the number of ways to make target sum at index **i**.

2. Overlapping Subproblems: While applying a recursive approach in this problem, we notice that certain subproblems are computed multiple times.

Follow the below steps to Implement the idea:

- Since there are **two parameters** that change during recursive calls, we create a **2D memo** array to store the results of previously solved subproblems.
- Since the range of recursion parameters goes from **0 to n** and **0 to sum**, we keep dimensions of the **2D array** as **(n+1) x (sum + 1)**
- **memo[i][j]** will represent the number of **distinct** ways to make the **sum j** by using the first **i coins**.
- During the recursion call, if the same state is called more than once, then we can directly return the answer stored for that state instead of calculating again.

C++ Java Python C# JavaScript

```
// C++ program for coin change problem
// using memoization
#include <bits/stdc++.h>
using namespace std;

// Returns the count of ways we can
// sum coins[0...n-1] coins to get sum "sum"
int countRecur(vector<int>& coins, int n, int sum,
               vector<vector<int>> &memo) {

    // If sum is 0 then there is 1 solution
    // (do not include any coin)
    if (sum == 0) return 1;

    // 0 ways in the following two cases
    if (sum < 0 || n == 0) return 0;

    // If the subproblem is previously calculated then
```

```

// simply return the result
if (memo[n-1][sum] != -1) return memo[n-1][sum];

// count is sum of solutions (i)
// including coins[n-1] (ii) excluding coins[n-1]
return memo[n-1][sum] =
    countRecur(coins, n, sum - coins[n-1], memo) +
    countRecur(coins, n - 1, sum, memo);
}

int count(vector<int> &coins, int sum) {

    vector<vector<int>> memo(coins.size(), vector<int>(sum+1,
-1));
    return countRecur(coins, coins.size(), sum, memo);
}

int main() {
    vector<int> coins = {1, 2, 3};
    int sum = 5;
    cout << count(coins, sum);
    return 0;
}

```

Output

5

[Better Approach 2] Using Bottom-Up DP (Tabulation) – $O(\text{sum} \times n)$ time and $O(\text{sum} \times n)$ space

The idea is to fill the **DP table** based on **previous** values. For each coin, we either **include** it or **exclude** it to compute the **minimum** number of coins needed for each sum. The table is filled in an iterative manner from $i = 1$ to $i = n$ and for each sum from 0 to sum.

The dynamic programming relation is as follows:

- if $(\text{sum} - \text{coins}[i-1])$ is **greater** than 0, then $dp[i][\text{sum}] = dp[i][\text{sum} - \text{coins}[i-1]] + dp[i+1][\text{sum}]$.

- $\text{else } dp[i][sum] = dp[i+1][sum].$

01
Step

Initialize a 2D DP table where $dp[i][j]$ represents the number of ways to make sum j using the first i coins.

Base Case: $dp[0][0] = 1$: One way to make the sum 0 - by not using any coins.
For all other sums ($dp[0][j]$), ways to make that sum is 0, no coins available.

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0

Coin Change – Count Ways to Make Sum

02
Step

For each coin and for each possible sum from 0 to the target sum:

Exclude the coin: Ways to make sum j without using the current coin is $dp[i-1][j]$.

Include the coin: If $j - \text{coins}[i-1] \geq 0$, Ways to make the sum j using the current coin is $dp[i][j - \text{coins}[i-1]]$.

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0

Coin Change – Count Ways to Make Sum

03
Step

First Coin (1): $dp[1][0] = dp[0][0]$, there is exactly one way to make sum 0, which is by not using any coin.

$$dp[i][j] = dp[i-1][j] + dp[i][j - \text{coins}[i-1]]$$

if $((j - \text{coins}[i-1]) \geq 0)$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0

Coin Change – Count Ways to Make Sum

04
Step

$dp[1][1] = dp[0][1] + dp[1][0]$, using one coin of denomination 1.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j - \text{coins}[i-1]]}_{\text{if } ((j - \text{coins}[i-1]) \geq 0)}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	0	0	0	0	0	0
3	0	0	0	0	0	0

$dp[1][2] = 1$: using two coin of denomination 1
 $dp[1][3] = 1$: using three coins of denomination 1.
 $dp[1][4] = 1$: using four coins of denomination 1.
 $dp[1][5] = 1$: using five coins of denomination 1.

Coin Change – Count Ways to Make Sum

05
Step

Second Coin (2): $dp[2][0] = dp[1][0]$, the number of ways to make sum 0 remains the same as before, which is by not using any coin.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j - \text{coins}[i-1]]}_{\text{if } ((j - \text{coins}[i-1]) \geq 0)}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	0	0	0	0
3	0	0	0	0	0	0

$dp[2][1] = dp[1][1]$
 The number of ways to make sum 1 remains the same as before, since the coin 2 cannot be used to make sum 1

Coin Change – Count Ways to Make Sum

06
Step

$dp[2][2] = dp[1][2] + dp[2][0]$.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j - \text{coins}[i-1]]}_{\text{if } ((j - \text{coins}[i-1]) \geq 0)}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	0	0	0
3	0	0	0	0	0	0

We can make sum 2 in two ways

- Two coins of denomination 1.
- One coin of denomination 2.

Coin Change – Count Ways to Make Sum

07
Step

$$dp[2][3] = dp[1][3] + dp[2][1].$$

 $dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	0	0
3	0	0	0	0	0	0

$$dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]]$$

if $((j - coins[i-1]) \geq 0)$

We can make sum 3 in two ways

- Three coins of denomination 1.
- One coin of denomination 2 and one coin of denomination 1.

Coin Change – Count Ways to Make Sum

08
Step

$$dp[2][4] = dp[1][4] + dp[2][2].$$

 $dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	0
3	0	0	0	0	0	0

$$dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]]$$

if $((j - coins[i-1]) \geq 0)$

We can make sum 4 in three ways:

- Four coins of denomination 1.
- Two coins of denomination 2.
- One coin of denomination 2 and two coins of denomination 1.

Coin Change – Count Ways to Make Sum

09
Step

$$dp[2][5] = dp[1][5] + dp[2][3].$$

 $dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	0	0	0	0	0	0

$$dp[i][j] = dp[i-1][j] + dp[i][j-coins[i-1]]$$

if $((j - coins[i-1]) \geq 0)$

We can make sum 5 in three ways:

- Five coins of denomination 1.
- One coin of denomination 2 and three coins of denomination 1.
- Two coins of denomination 2 and one coin of denomination 1.

Coin Change – Count Ways to Make Sum

10
Step

Third Coin (3): $dp[3][0] = dp[2][0]$, the number of ways to make sum 0 remains the same as before, which is by not using any coin.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j-coins[i-1]]}_{\text{if } (j - coins[i-1]) \geq 0}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	0	0	0

$dp[3][1] = dp[2][1]$: The number of ways to make sum 1 remains the same as before, since coin 3 cannot be used to make sum 1.

$dp[3][2] = dp[2][2]$: The number of ways to make sum 2 remains the same as before, since coin 3 cannot be used to make sum 2.

Coin Change – Count Ways to Make Sum

11
Step

$dp[3][3] = dp[2][3] + dp[3][0]$.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j-coins[i-1]]}_{\text{if } (j - coins[i-1]) \geq 0}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	3	0	0

We can make sum 3 in three ways

- Three coins of denomination 1.
- One coin of denomination 2 and one coin of denomination 1.
- One coin of denomination 3.

Coin Change – Count Ways to Make Sum

12
Step

$dp[3][4] = dp[2][4] + dp[3][1]$.

$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j-coins[i-1]]}_{\text{if } (j - coins[i-1]) \geq 0}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	3	4	0

We can make sum 4 in four ways

- Four coins of denomination 1.
- Two coins of denomination 2.
- One coin of denomination 2 and two coins of denomination 1.
- One coin of denomination 3 and one coin of denomination 1.

Coin Change – Count Ways to Make Sum

13
Step $dp[3][5] = dp[2][5] + dp[3][2].$
$$dp[i][j] = dp[i-1][j] + \underbrace{dp[i][j-coins[i-1]]}_{\text{if } (j - coins[i-1]) \geq 0}$$

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	3	4	5

We can make sum 5 in five ways

- Five coins of denomination 1.
- One coin of denomination 2 and three coins of denomination 1.
- Two coins of denomination 2 and one coin of denomination 1.
- One coin of denomination 3 and two coins of denomination 1.
- One coin of denomination 3 and one coin of denomination 2.

14
StepThe number of ways to make sum 5 using coins {1, 2, 3} is $dp[3][5]$.

$dp[][] =$

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	3	4	5

C++

Java

Python

C#

JavaScript

// C++ program for coin change problem using tabulation

#include <bits/stdc++.h>

using namespace std;

// Returns total distinct ways to make sum using n coins of
// different denominations

int count(vector<int>& coins, int sum) {

int n = coins.size();

// 2d dp array where n is the number of coin

// denominations and sum is the target sum

vector<vector<int>> dp(n + 1, vector<int>(sum + 1, 0));

// Represents the base case where the target sum is 0,

// and there is only one way to make change: by not

// selecting any coin

dp[0][0] = 1;

for (int i = 1; i <= n; i++) {

for (int j = 0; j <= sum; j++) {

```

        // Add the number of ways to make change without
        // using the current coin,
        dp[i][j] += dp[i - 1][j];

        if ((j - coins[i - 1]) >= 0) {

            // Add the number of ways to make change
            // using the current coin
            dp[i][j] += dp[i][j - coins[i - 1]];
        }
    }
}

return dp[n][sum];
}

int main() {
    vector<int> coins = {1, 2, 3};
    int sum = 5;
    cout << count(coins, sum);
    return 0;
}

```

Output

5

[Expected Approach] Using Space Optimized DP – $O(\text{sum} \cdot n)$ time and $O(\text{sum})$ space

In **previous approach** of dynamic programming we have derive the relation between states as given below:

- if $(\text{sum} - \text{coins}[i])$ is greater than 0, then $dp[i][\text{sum}] = dp[i][\text{sum} - \text{coins}[i]] + dp[i+1][\text{sum}]$.
- else $dp[i][\text{sum}] = dp[i+1][\text{sum}]$.

If we observe that for calculating **current** $dp[i][\text{sum}]$ state we only need previous row $dp[i-1][\text{sum}]$ or current row $dp[i][\text{sum} - \text{coins}[i]]$. There is no

need to store all the **previous** states just one previous state is used to compute result.

C++ Java Python C# JavaScript

```
// C++ program for coin change problem
// using space optimised dp
#include <bits/stdc++.h>
using namespace std;

// Returns total distinct ways to make sum using n coins of
// different denominations
int count(vector<int> &coins, int sum) {
    int n = coins.size();

    // dp[i] will be storing the number of solutions for
    // value i. We need sum+1 rows as the dp is
    // constructed in bottom up manner using the base case
    // (sum = 0)
    vector<int> dp(sum + 1);

    // Base case (If given value is 0)
    dp[0] = 1;

    // Pick all coins one by one and update the table[]
    // values after the index greater than or equal to the
    // value of the picked coin
    for (int i = 0; i < n; i++)
        for (int j = coins[i]; j <= sum; j++)
            dp[j] += dp[j - coins[i]];
    return dp[sum];
}

int main() {
    vector<int> coins = {1, 2, 3};
    int sum = 5;
    cout << count(coins, sum);
    return 0;
}
```

Output

5

Related articles:

- [Coin Change – Minimum Coins to Make Sum](#)
- [Understanding The Coin Change Problem With Dynamic Programming](#)

Coin Change – Count Ways to Make Sum

 Comment



kartik

+ Follow

 526



Article Tags :

Dynamic Programming

Greedy

Mathematical

DSA

+6 More

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information